
Adversarial Robustness

Chimirela Karthikeshwar

K Kausigan

Saifilkhan Pathan

Shubham Harikrishnabhai Meghani

Sudarshana Chaitanya B N

Abstract

Adversarial robustness remains a critical challenge for modern machine learning systems. This work investigates two directions toward understanding and improving robustness in neural networks. First, we study asymmetric adversarial transferability across architectures, finding that gradient norm differences are the dominant predictor of transfer asymmetry. Second, we explore combining diffusion-based synthetic data with Incremental Adversarial Training, finding that Fisher regularization preserves clean accuracy but constrains the parameter flexibility needed for adversarial robustness. Together, these findings offer concrete insights into transferability and the trade-offs inherent in knowledge-preserving defense strategies.

1 Introduction

Deep learning models achieve strong performance in tasks such as computer vision and natural language processing, yet they remain vulnerable to adversarial perturbations i.e. small input changes that can cause incorrect predictions. These vulnerabilities raise concerns about the reliability of machine learning systems, particularly in safety-critical applications.

Adversarial robustness studies how models behave under such perturbations and seeks methods that maintain reliable predictions when inputs are manipulated. A common defense is *adversarial training*, where models are trained on adversarially perturbed inputs. For a classifier f_θ with parameters θ , adversarial training is typically formulated as

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\max_{\|\delta\| \leq \epsilon} \mathcal{L}(f_\theta(x + \delta), y) \right], \quad (1)$$

where (x, y) is sampled from distribution $\mathcal{D}$, δ is a bounded perturbation, and $\mathcal{L}$ is the training loss. The inner maximization finds worst-case perturbations, while the outer minimization learns parameters robust to them.

Despite progress, achieving reliable robustness remains challenging. Adversarial examples can transfer across models, and models may also be vulnerable to other threats such as backdoor attacks.

In this work, we explore several directions for improving robustness: analyzing model properties that influence asymmetric adversarial transferability, studying defenses against both adversarial and backdoor attacks, and combining diffusion-based purification with incremental adversarial training.

2 Model Properties Driving Asymmetric Adversarial Transferability

2.1 Motivation

In this section, we study adversarial transferability across different neural network architectures and investigate why it is often asymmetric, building on prior work by Adam et al. (3).

2.2 Experimental Setup

We study adversarial transferability across five architectures: ResNet18, ResNet34, VGG11, VGG16, and DenseNet121, all trained on CIFAR-10 to ensure fair comparison.

Training. All models are trained using cross-entropy loss with the Adam optimizer. Standard data augmentation is applied (random cropping and horizontal flipping). Models are trained for 50 epochs, with a learning rate of 10^{-3} for ResNets and DenseNet, and 10^{-4} for VGG models (to ensure stable optimization given their higher parameter count and sensitivity to larger step sizes).

Adversarial Attacks. Adversarial examples are generated using Projected Gradient Descent (PGD), as proposed by Madry et al. (4), under an ℓ_∞ constraint:

$$\|\delta\|_\infty \leq \epsilon, \quad \epsilon = \frac{8}{255},$$

with step size $\alpha = \frac{2}{255}$ and 20 iterations. Random initialization within the ϵ -ball is used.

For each source model f_s , adversarial examples are generated and evaluated on a target model f_t . The loss function $\mathcal{L}$ used for both training and attack generation is the cross-entropy loss.

Fooling Rate (Transferability Metric).

$$\text{FR}(f_s \rightarrow f_t) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(f_t(x_i^{\text{adv}}) \neq y_i)$$

This measures the fraction of adversarial examples generated on source model f_s that successfully fool target model f_t . Equivalently, it is $1 - \text{accuracy}$ of the target model on adversarial inputs.

Asymmetry.

$$\Delta(f_s, f_t) = |\text{FR}(f_s \rightarrow f_t) - \text{FR}(f_t \rightarrow f_s)|$$

Model Properties. To analyze asymmetry, we compute:

- **Gradient norms:** Average ℓ_2 norm of input gradients $\|\nabla_x \mathcal{L}(f(x), y)\|_2$, computed per sample, flattened, and averaged.
- **Gradient cosine similarity:** Average cosine similarity between input gradients of two models, computed per sample and averaged.
- **Parameter count:** Total number of trainable parameters in each model.

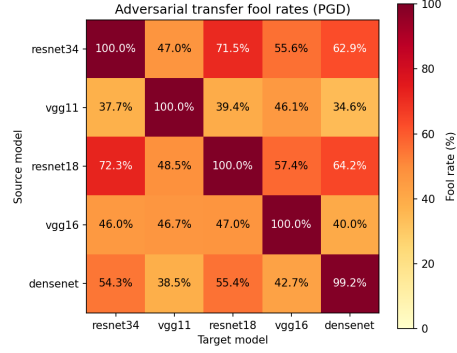
Gradients are computed with respect to input images and evaluated on a subset of the test set (50 batches) for computational efficiency.

Model	Params (M)	Grad Norm	Accuracy (%)
ResNet18	11.18	0.0865	85.58
ResNet34	21.29	0.0865	85.61
VGG11	128.81	0.1343	82.48
VGG16	134.30	0.1337	85.08
DenseNet121	6.96	0.1172	86.32

Table 1: Model properties used in analysis. Gradient norms are averaged over samples.

2.3 Results

For most pairs, transferability is asymmetric. ResNet models often generate perturbations that transfer well to other architectures, while the reverse transfer is weaker (e.g., $\text{ResNet} \rightarrow \text{VGG} > \text{VGG} \rightarrow \text{ResNet}$). DenseNet exhibits highly inconsistent behavior, with significant variation across different model pairs.



Model Pair	Norm Diff	Cos Sim	Asym (%)
ResNet34–VGG11	0.0477	0.0962	9.28
ResNet34–ResNet18	0.0000	0.2226	0.88
ResNet34–VGG16	0.0472	0.1187	9.53
ResNet34–DenseNet	0.0307	0.1548	8.56
VGG11–ResNet18	0.0477	0.1007	9.15
VGG11–VGG16	0.0005	0.0972	0.59
VGG11–DenseNet	0.0170	0.0748	3.86
ResNet18–VGG16	0.0472	0.1195	10.34
ResNet18–DenseNet	0.0307	0.1549	8.87
VGG16–DenseNet	0.0165	0.0864	2.75

Table 2: Relationship between gradient properties and transfer asymmetry.

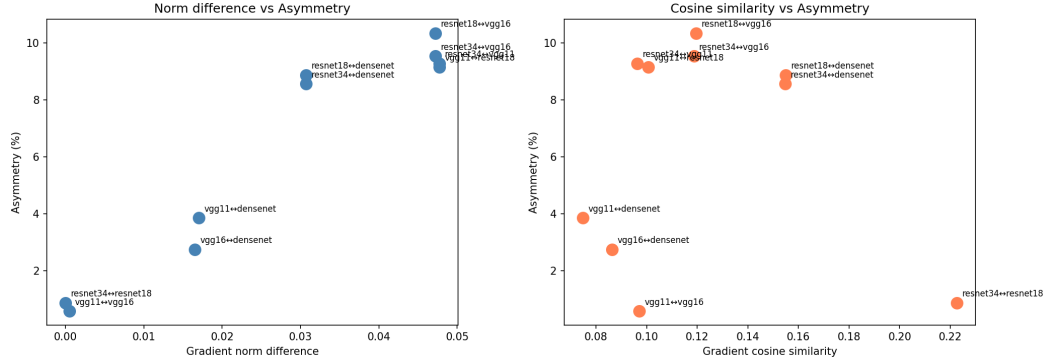


Figure 1: Relationship between model properties and transfer asymmetry. Gradient norm differences show a strong positive linear relationship with asymmetry, while cosine similarity shows no clear trend.

Metric	Pearson (r)	Spearman (ρ)	p -value
Gradient Norm Difference vs. Asym	0.9580	0.8788	$< 10^{-3}$
Cosine Similarity vs. Asym	-0.1049	0.1030	> 0.7
Log Parameter Ratio vs. Asym	0.2541	0.3091	> 0.3

Table 3: Correlation analysis between model properties and transfer asymmetry.

2.4 Analysis

To explain these observations, we follow the first-order analysis of adversarial perturbations introduced by Goodfellow et al. (5) and further studied in the context of transferability by Adam et al. (3). Using Taylor approximation:

$$\mathcal{L}(x + \delta) \approx \mathcal{L}(x) + \delta^\top \nabla_x \mathcal{L}(x).$$

For PGD-style attacks:

$$\delta = \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}_s(x)).$$

Substituting into the target model:

$$\mathcal{L}_t(x + \delta) \approx \mathcal{L}_t(x) + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}_s(x))^\top \nabla_x \mathcal{L}_t(x).$$

$$\Delta \mathcal{L}_t \approx \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}_s(x))^\top \nabla_x \mathcal{L}_t(x),$$

Comparing directions:

$$f_s \rightarrow f_t : \quad \Delta \mathcal{L}_t \propto \|\nabla_x \mathcal{L}_t(x)\|, \quad f_t \rightarrow f_s : \quad \Delta \mathcal{L}_s \propto \|\nabla_x \mathcal{L}_s(x)\|.$$

The norm term depends on which model is the target. Thus, if

$$\|\nabla_x \mathcal{L}_t(x)\| \neq \|\nabla_x \mathcal{L}_s(x)\|,$$

then

$$\text{FR}(f_s \rightarrow f_t) \neq \text{FR}(f_t \rightarrow f_s),$$

leading to directional asymmetry.

Cosine similarity is symmetric, so intuitively it should not relate to asymmetry:

$$\cos(\nabla_s, \nabla_t) = \cos(\nabla_t, \nabla_s).$$

These theoretical insights align with the empirical results. Model pairs with larger gradient norm differences consistently exhibit higher asymmetry, while cosine similarity shows no meaningful relationship.

Conclusion.

- Gradient norm difference strongly correlates with asymmetry.
- Gradient direction (cosine similarity) does not influence asymmetry.
- Number of parameters has weak influence.

3 Combining Diffusion Models with Incremental Adversarial Training

3.1 How Better Diffusion Models Help in Adversarial Training

As we read in the paper (1), we understand that improved diffusion models enhance adversarial training by generating high-quality synthetic data that better approximates the true data distribution. This reduces robust overfitting, improves generalization, and narrows the gap between clean and robust accuracy. Both the quality (low FID) and quantity of generated data are crucial, as they enrich the training distribution and strengthen decision boundaries against adversarial attacks.

3.2 Incremental Adversarial Training (IncAT)

As we read in the paper (2), we understand that IncAT improves adversarial training by **incrementally updating** the model instead of retraining from scratch. The key idea is to preserve important knowledge from clean data while learning adversarial examples. This is achieved using the **Fisher Information Matrix (FIM)**, which measures the importance of each parameter. Parameters with higher importance are constrained from changing significantly during adversarial training.

The method introduces an Elastic Weight Consolidation (EWC) regularization into the loss:

$$L(\theta) = L_B(\theta) + \sum_i \frac{\lambda}{2} F_i (\theta_i - \theta_{A,i}^*)^2,$$

where $L_B(\theta)$ is the adversarial loss, F_i denotes the Fisher importance of parameter θ_i , $\theta_{A,i}^*$ are the original model parameters, and λ controls the trade-off between retaining old knowledge and learning adversarial features.

This formulation ensures that important parameters (high F_i) receive larger penalties if changed, allowing the model to **retain clean-data knowledge while adapting to adversarial samples**. As shown in the training pipeline (Fig. 1, page 3), the model first learns from clean data, computes FIM, and then incrementally incorporates adversarial samples with constrained updates.

3.3 Experiment Setup

The experiment was conducted on the CIFAR-10 dataset using a ResNet-18 model trained from scratch. The training data consisted of both real and synthetic samples, with 30% and 70% of the data drawn from an auxiliary synthetic dataset to enable mixed training.

Baseline training was performed using TRADES for 50 and 100 epochs. Adversarial examples were generated using L_∞ -bounded PGD with $\epsilon = 8/255$, step size $2/255$, and 2, 8, or 10 iterations depending on the run. Optimization used SGD with Nesterov momentum, learning rate 0.02, weight decay 5×10^{-4} , cosine scheduling, and gradient clipping.

Following this, Incremental Adversarial Training (IncAT) was applied as a fine-tuning stage. The model was optimized using adversarial loss with an added Fisher regularization term, which penalized deviations from previously learned parameters based on their estimated importance.

Evaluation was carried out using AutoAttack. The reported metrics include clean accuracy and robust accuracy under adversarial perturbations.

3.4 Results

Experiment	Syn	Ep	PGD	ϵ	λ	Clean	Robust
TRADES	30	50	2	0.031	—	80.27	29.96
IncAT	30	+50	10	0.031	10^{-5}	74.77	36.08
TRADES	70	100	8	0.031	—	85.16	52.26
IncAT	70	+50	10	0.031	10^{-5}	80.43	47.44

Table 4: Robust accuracy results on CIFAR-10 with ResNet-18.

3.5 Analysis

The TRADES baseline achieved strong clean performance and moderate robustness. However, IncAT fine-tuning with Fisher regularization led to a clear drop in robust accuracy while leaving clean accuracy nearly unchanged.

The degradation appears early in the attack sequence (APGD-CE), indicating increased vulnerability to standard gradient-based attacks. This suggests that the model’s ability to adapt its decision boundary was reduced during fine-tuning.

The Fisher penalty successfully constrained parameter updates, preserving clean accuracy, but this constraint likely limited the flexibility required for effective adversarial training. As a result, robustness was negatively impacted.

The combination of TRADES with IncAT and Fisher regularization did not improve adversarial robustness in this setting. While clean accuracy was preserved, robust accuracy decreased significantly. These results indicate that constraining parameter updates through Fisher regularization may interfere with the boundary adjustments required for adversarial robustness.

References

- [1] Zekai Wang et al., *Better Diffusion Models Further Improve Adversarial Training*, ICML, 2023.
- [2] Yuxin Ge et al., *An Incremental Adversarial Training Method Enables Timeliness and Rapid New Knowledge Acquisition*, Scientific Reports, 2025.
- [3] George Adam, Petr Smirnov, Benjamin Haibe-Kains, Anna Goldenberg, *Reducing Adversarial Example Transferability Using Gradient Regularization*, 2021.
- [4] Aleksander Madry et al., *Towards Deep Learning Models Resistant to Adversarial Attacks*, ICLR, 2018.
- [5] Ian J. Goodfellow et al., *Explaining and Harnessing Adversarial Examples*, ICLR, 2015.
- [6] Florian Tramèr et al., *The Space of Transferable Adversarial Examples*, IEEE Symposium on Security and Privacy, 2017.
- [7] Lei Wu et al., *Understanding and Enhancing the Transferability of Adversarial Examples*, arXiv:1802.09707, 2018.
- [8] Hongyang Zhang et al., *Theoretically Principled Trade-off between Robustness and Accuracy (TRADES)*, ICML, 2019.
- [9] Jacob Clarysse et al., *Why Adversarial Training Can Hurt Robust Accuracy*, arXiv:2203.02006, 2022.

A Appendix

A.1 Gradient Definition: Model vs Loss

Adam et al. (3) analyze gradients of the model output, whereas in our experiments we compute gradients of the loss function $\nabla_x \mathcal{L}(f(x), y)$.

For standard classification losses such as cross-entropy, the two are related via the chain rule:

$$\nabla_x \mathcal{L}(f(x), y) = \frac{\partial \mathcal{L}}{\partial f} \cdot \nabla_x f(x).$$

Thus, the loss gradient is a scaled version of the model gradient. This scaling does not affect gradient direction and introduces only a multiplicative factor in magnitude. Since our analysis focuses on relative norm differences and alignment, this does not qualitatively change the observed relationships.

A.2 Projected Gradient Descent (PGD) Attack

Starting from an initial perturbation $x_0 = x + \delta_0$ with $\|\delta_0\|_\infty \leq \epsilon$, PGD performs the update:

$$x_{k+1} = \Pi_{\mathcal{B}_\epsilon(x)} \left(x_k + \alpha \cdot \text{sign} \left(\nabla_x \mathcal{L}(f(x_k), y) \right) \right),$$

where $\Pi_{\mathcal{B}_\epsilon(x)}$ denotes projection onto the ℓ_∞ ball of radius ϵ around the original input x .

This formulation makes explicit that adversarial perturbations are constructed using the gradient of the loss function with respect to the input, which is consistent with the gradient-based analysis presented in this work.

A.3 Why Cosine Similarity Does Not Explain Asymmetry

Cosine similarity between gradients is symmetric:

$$\cos(\nabla_s, \nabla_t) = \cos(\nabla_t, \nabla_s).$$

However, transferability depends on the directional derivative:

$$\text{sign}(\nabla_s)^\top \nabla_t,$$

which is not symmetric due to dependence on the target gradient magnitude. Therefore, even if gradients are well aligned, differences in gradient norms can produce asymmetric transfer.

A.4 Subset Evaluation

Gradient statistics and transferability are computed on a subset of the test set (50 batches) for computational efficiency. Since all model pairs are evaluated on the same subset, relative comparisons remain consistent. The observed trends—particularly the strong correlation between gradient norm differences and asymmetry—are robust to such subsampling.

A.5 Diagonal Fisher Information via Cross-Entropy Loss

Definition of Fisher Information For a parametric model $p_\theta(y | x)$, the Fisher Information Matrix is defined as:

$$F_{ij} = \mathbb{E}_{(x,y) \sim p_{\text{data}}} \left[\frac{\partial \log p_\theta(y|x)}{\partial \theta_i} \frac{\partial \log p_\theta(y|x)}{\partial \theta_j} \right]$$

The diagonal entries are:

$$F_{ii} = \mathbb{E}_{(x,y) \sim p_{\text{data}}} \left[\left(\frac{\partial \log p_\theta(y|x)}{\partial \theta_i} \right)^2 \right]$$

Cross-Entropy Loss For classification, the cross-entropy loss is defined as:

$$\mathcal{L}_{CE}(x, y; \theta) = -\log p_\theta(y|x)$$

Gradient Relationship Differentiating with respect to θ_i :

$$\frac{\partial \mathcal{L}_{CE}}{\partial \theta_i} = - \frac{\partial \log p_\theta(y|x)}{\partial \theta_i}$$

Rewriting:

$$\frac{\partial \log p_\theta(y|x)}{\partial \theta_i} = - \frac{\partial \mathcal{L}_{CE}}{\partial \theta_i}$$

Step 4: Substitution into Fisher Substituting into the diagonal Fisher expression:

$$F_{ii} = \mathbb{E} \left[\left(- \frac{\partial \mathcal{L}_{CE}}{\partial \theta_i} \right)^2 \right]$$

$$F_{ii} = \mathbb{E} \left[\left(\frac{\partial \mathcal{L}_{CE}}{\partial \theta_i} \right)^2 \right]$$

Empirical Estimate In practice, the expectation is approximated using a dataset of size N :

$$F_{ii} \approx \frac{1}{N} \sum_{n=1}^N \left(\frac{\partial \mathcal{L}_{CE}(x_n, y_n; \theta)}{\partial \theta_i} \right)^2$$

Final Result

$$F_{ii} = \mathbb{E}_{(x,y) \sim p_{\text{data}}} \left[\left(\frac{\partial \mathcal{L}_{CE}(x, y; \theta)}{\partial \theta_i} \right)^2 \right]$$

A.6 Code for all the experiments conducted

Here is the link to all the experiments conducted <https://github.com/KausiganK/Adversarial-Robustness>